

Simple Blog Platform for Sharing and Managing Technical Posts

Informal User Story

As a small technical community, we want a simple blog platform where visitors can read public content, registered users can contribute posts, and administrators can organize content categories.

The goal of the application is not to build a large CMS, but to demonstrate a realistic full-stack workflow using a clean backend architecture, a clear domain model, authentication, authorization, business rules, and CRUD operations.

Anonymous visitors can browse public blog posts and interact with them by liking or disliking content. Users who create an account can write their own posts, edit or remove only the posts they own, and create tags to help organize content. Administrators have additional permissions to manage post categories, making sure the blog content remains organized.

The application should clearly separate public actions from protected actions. Reading posts and reacting to posts should be available publicly, while creating, editing, deleting posts, creating tags, and managing categories should require authentication or specific authorization.

This project is designed to show how a small but complete product can be implemented with a .NET backend, a React frontend, business validation, custom data access, unit tests, and a structure that is easy to explain during a technical interview.

Main Actors

Anonymous Visitor

A user who is not logged in.

They can access public content and interact with posts using simple reactions.

Main actions:

- Read public posts
- Like or dislike public posts
- Register for an account
- Log in

Authenticated User / Guest User

A registered user who is logged in.

They can contribute content and manage only their own posts.

Main actions:

- Read public posts.
- Create posts.
- Edit their own posts.
- Remove their own posts.
- Create tags.
- Like or dislike posts.
- Log out.

Administrator

A privileged user responsible for organizing blog content.

Main actions:

- Read public posts.
- Manage post categories.
- Perform actions available to authenticated users, depending on permissions.
- Access administrator-only functionality.

MVP Scope

The MVP is a small blog application focused on the core features required to demonstrate full-stack development.

Included in the MVP:

- User registration and login.
- Public post listing.
- Public post details.
- Create, read, update, and delete operations for posts.
- Authorization rules for editing and removing posts.
- Basic like/dislike functionality.

- Tag creation by authenticated users.
- Category management by administrators.
- React frontend integrated with the [ASP.NET](#) Core Web API.
- Seeded demo users and sample blog content.
- Unit tests for business rules, data access behavior, and API endpoints.

The application should remain intentionally small. The focus is on clean implementation, readable code, validation, testing, and architectural decisions.

Primary User Flows

Flow 1: Anonymous visitor reads and reacts to posts

An anonymous visitor opens the blog homepage and sees a list of public posts. They can open a post details page to read the full content. After reading, they can like or dislike the post without needing to log in.

This flow demonstrates public API endpoints and frontend integration for read-only and lightweight interaction features.

Flow 2: User registers and logs in

A visitor creates an account using the registration form. After registration, they can log in and receive access to protected features.

This flow demonstrates user creation, authentication, credential validation, and storage of user information.

Flow 3: Authenticated user creates a post

After logging in, a user can create a blog post by providing the required content and selecting or creating tags. The new post becomes available in the public post list.

This flow demonstrates protected endpoints, business validation, data persistence, and frontend form handling.

Flow 4: Authenticated user edits their own post

A logged-in user opens one of their own posts and updates its content. The system verifies that the logged-in user is the owner of the post before allowing the update.

This flow demonstrates authorization based on ownership and business rules in the application layer.

Flow 5: Authenticated user removes their own post

A logged-in user can remove a post they created.

The system prevents them from removing posts created by other users.

This flow demonstrates protected delete operations and ownership validation.

Flow 6: Administrator manages post categories

An administrator logs in and accesses a simple category management screen.

They can create, update, or remove post categories used to organize blog posts.

This flow demonstrates role-based authorization and administrator-only functionality.

Business Rules

- Public posts can be read by anyone.
- Users must be authenticated to create posts.
- Users can edit only posts they created.
- Users can remove only posts they created.
- Administrators can manage post categories.
- Tags can be created by authenticated users.
- Like and dislike actions are available for public posts.
- A post should have enough required content to be considered valid.
- A post should belong to a valid category when categories are used.
- Protected endpoints must reject unauthenticated requests.
- Administrator endpoints must reject users without the required role or permission.
- Authentication and authorization rules must be enforced in the backend, not only in the frontend.
- Business validation should live outside the API controller layer.
- Data access should be abstracted so the business logic does not depend directly on storage implementation details.

Out of Scope

The following features are intentionally out of scope for the technical exercise:

- Rich text editor.
- Image upload.
- Comment system.
- Email confirmation.
- Password recovery.
- Advanced moderation workflow.
- Full CMS capabilities.
- Search engine optimization features.
- Post scheduling.
- Draft and publishing workflow.
- Notification system.
- Social sharing integration.
- Advanced analytics.
- Multi-tenant support.
- External identity providers.

These features could be added in a real product, but they would increase the scope beyond what is necessary for a focused technical interview project.

Why this project fits the technical challenge

This blog platform is small enough to be completed within a technical interview exercise, but complete enough to demonstrate the requested engineering skills.

It includes CRUD operations through an API, user registration and login, public and protected endpoints, business rules, authorization checks, a data model with multiple related concepts, and frontend integration with React.

The project also gives a clear reason to apply Clean Architecture. The API layer can handle HTTP concerns, the business layer can enforce rules such as post ownership and administrator permissions, and the data layer can provide custom persistence logic without relying on Entity Framework, Dapper, or Mediator.

The domain is also easy to explain during a code review. Interviewers can quickly understand posts, users, tags, categories, roles, permissions, and reactions without needing deep business context.

This makes the project a good balance between realistic application design and manageable implementation scope.

Short version for presentation

This project is a simple blog platform built to demonstrate a complete full-stack application using .NET and React.

Anonymous visitors can read public posts and react with likes or dislikes. Registered users can create posts, edit or remove only their own posts, and create tags. Administrators can manage post categories.

The goal is not to build a full CMS, but to show a clean and testable application structure with authentication, authorization, CRUD operations, business validation, custom data access, and frontend integration.

The project fits the technical challenge because it includes public and protected API endpoints, a clear data model, ownership-based business rules, role-based administrator functionality, unit tests, and a user interface that demonstrates the main use cases end to end.